Lecture 08

Design Metrics and CPU Performance Evaluation

Instructor: Dr. Muhammad Imran

CHIPXPRT

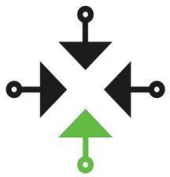
NCDC
NUST CHIP DESIGN CENTRE

Acknowledgement

- Content from following has been used in these lectures
  - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy
  - Computer Organization and Design, RISC-V 1st Edition, Patterson and Hennessy
  - CS 61C: Great Ideas in Computer Architecture (Machine Structures), UC Berkeley

Contents

Design Metrics and Design Tradeoffs



CHIPXPRT

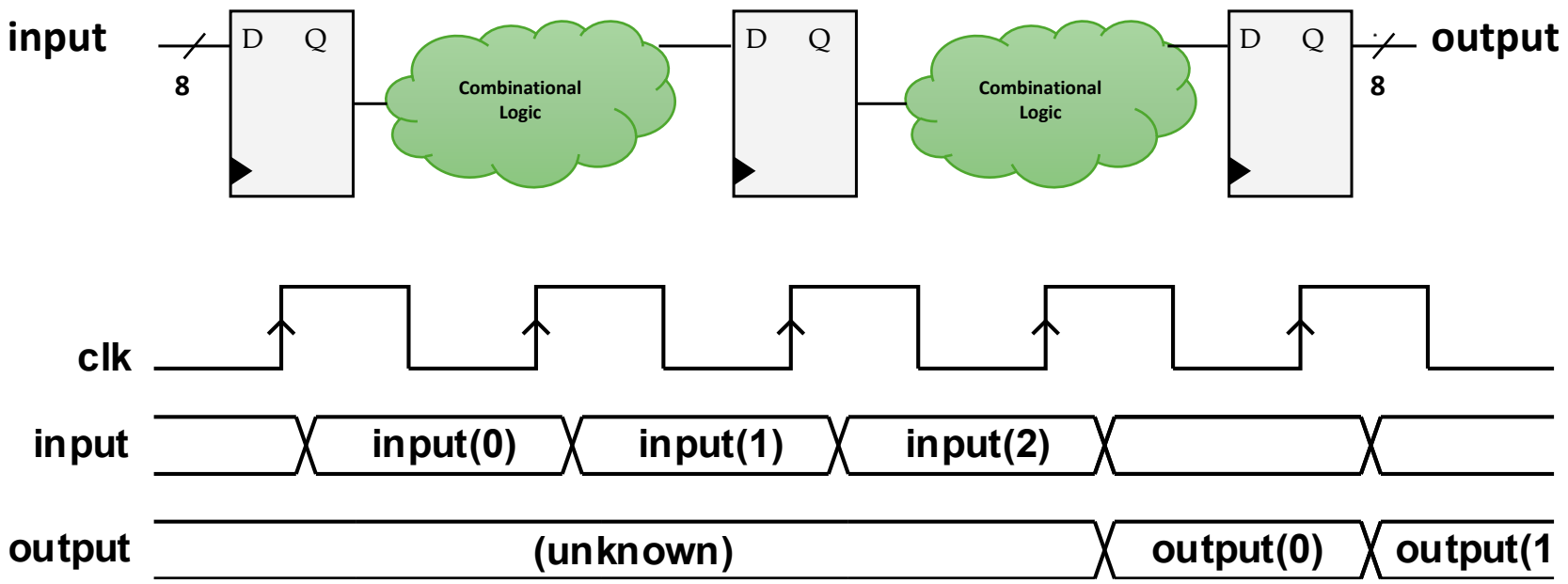
Measuring Speed of a Design

- Throughput
  - Amount of data processed per clock cycle
    - Bits per cycle or bits per second
  - Tasks executed per unit time
    - Instructions per second, Instructions per cycle etc.

- Latency
  - Time to process a single task
    - Number of cycles or seconds

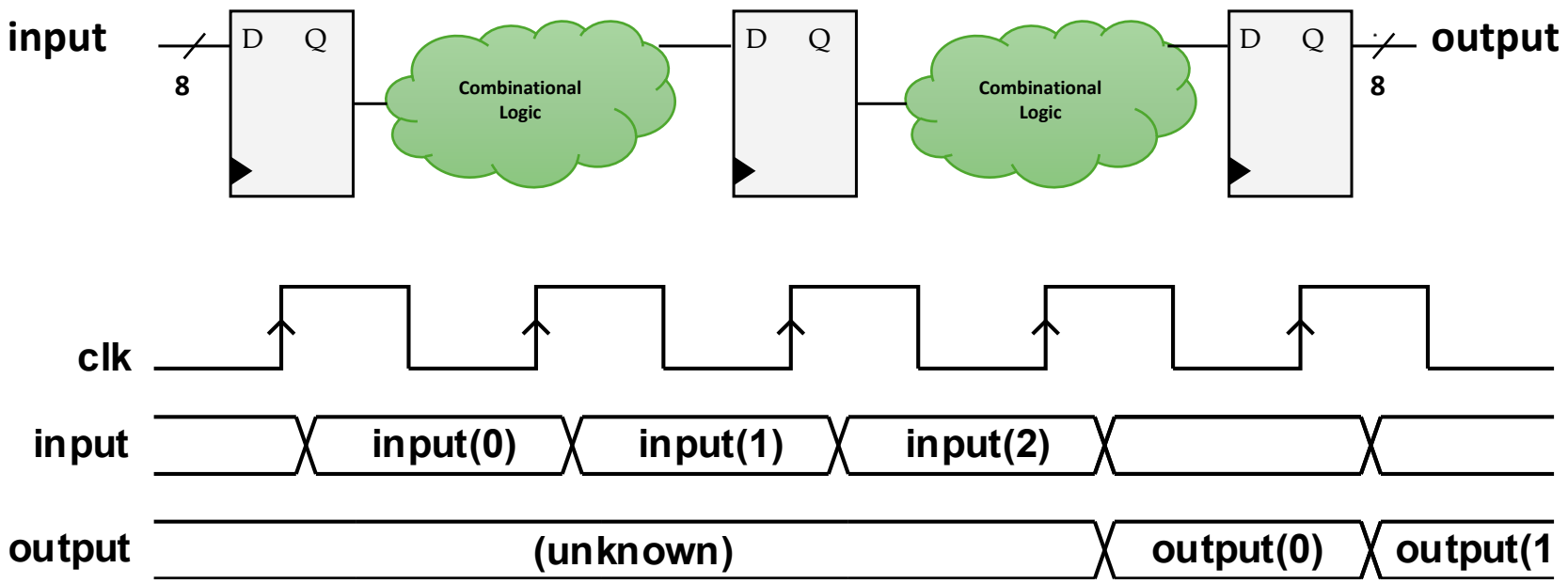
- Timing
  - Defined by the logic delays between sequential elements
    - Clock period, frequency

Example …



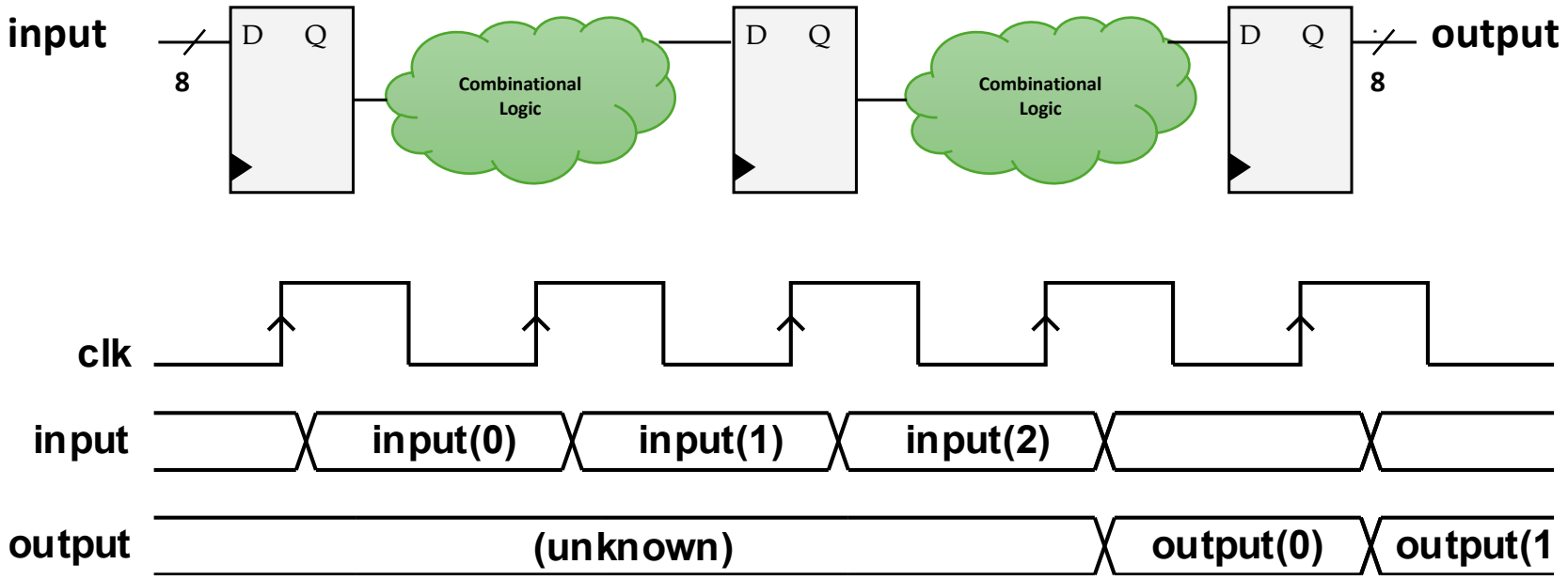
- Throughput?
  - (Bits per output sample / time between two output samples)
  - 8 bits/cycle, if 1 cycle = 10 ns, throughput = 8/10n = 800 Mbits/s
  - Throughput can also be 1 task/sample per cycle!!

Example …



- Latency?
  - Time to complete one task / sample
  - 3 clock cycles, if 1 cycle = 10 ns, latency = 30 ns

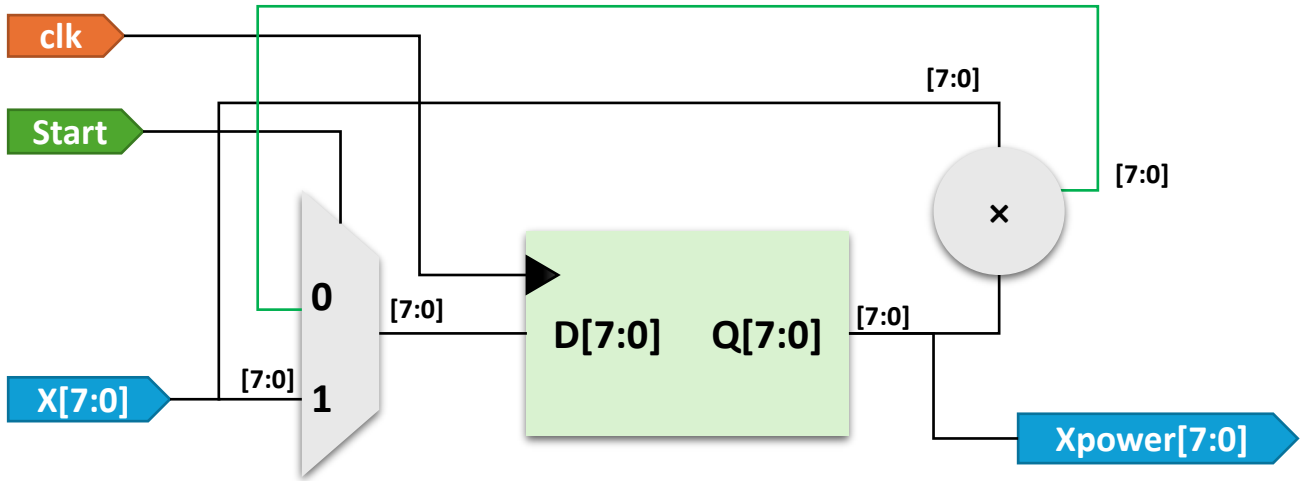
Example …



- Timing?
  - Clock period = tckl2q + combinational logic delay (longest) + ts

Design Tradeoffs: Multicycle Design

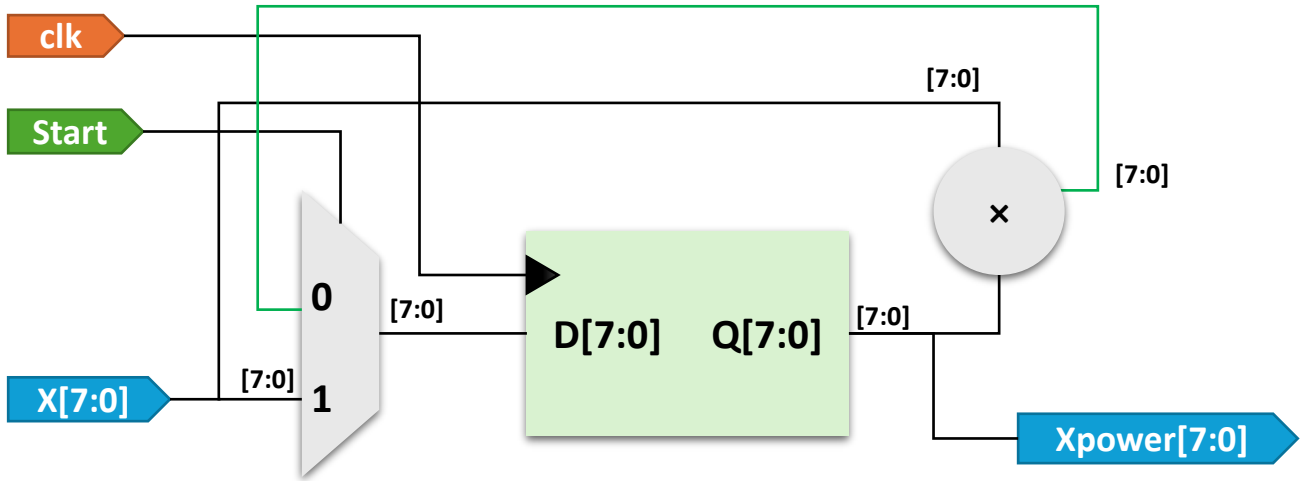
```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```



- Throughput?
  - 1 sample or task / 3 cycles
  - 8 bits / 3 cycles = 2.7 bits per

Design Tradeoffs: Multicycle Design

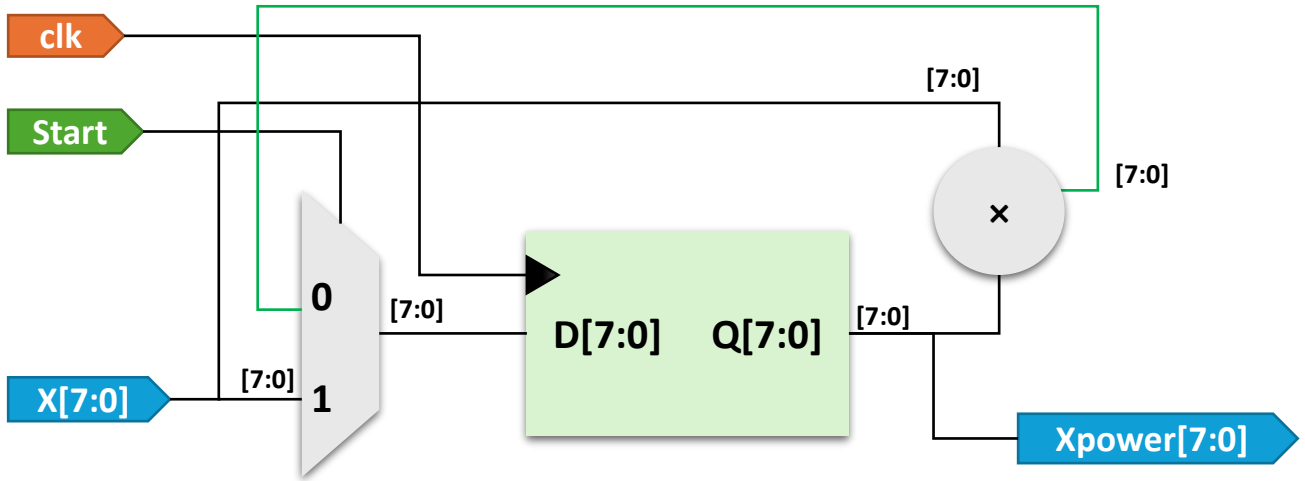
```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```



- Latency?
  - 3 clock cycles

Design Tradeoffs: Multicycle Design

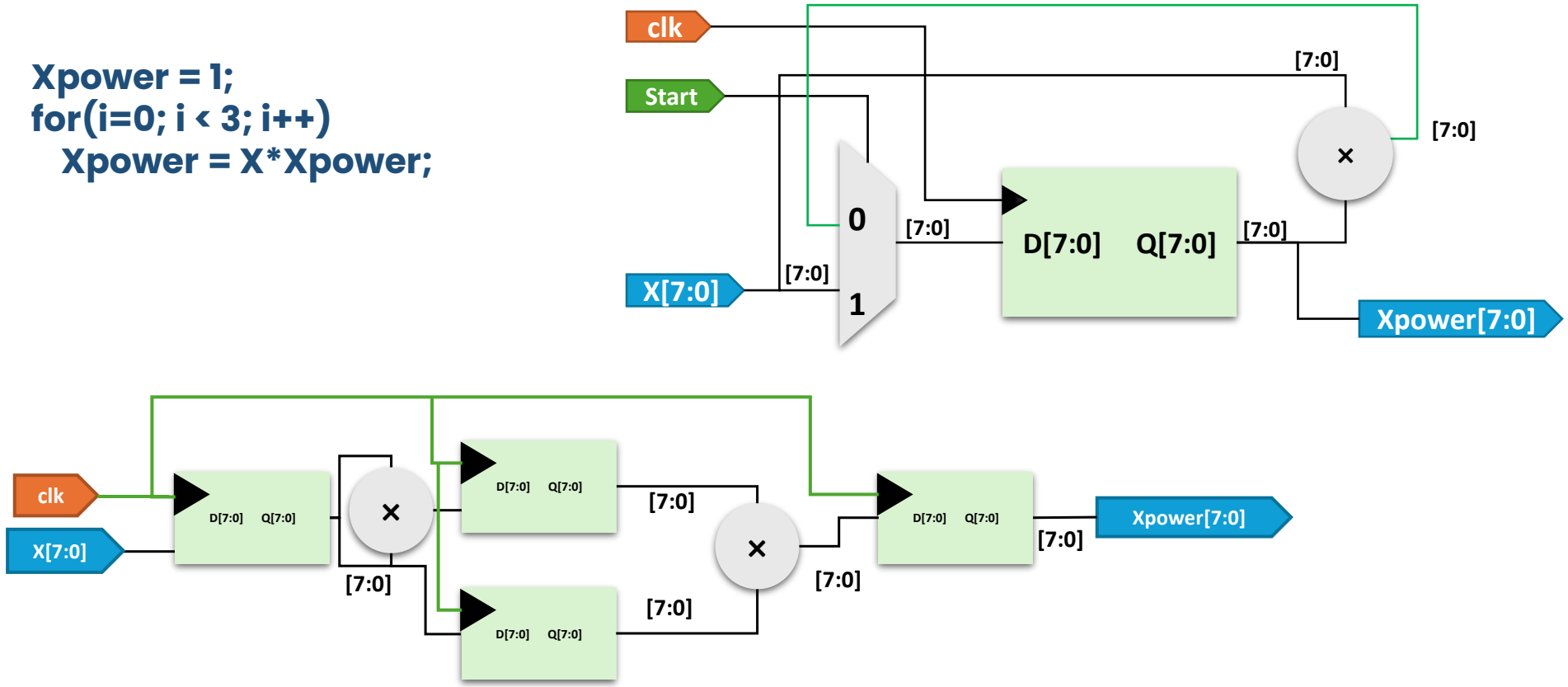
```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```



- Clock Timing?
  - Clock period = tclk2q + 1 multiplier delay + 1 mux delay + ts

Design Tradeoffs: Pipelining

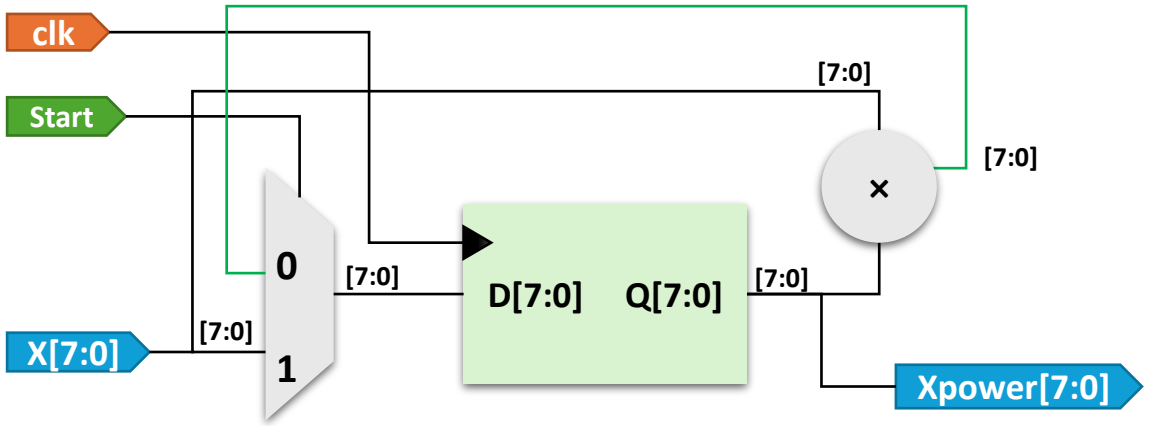
```
Xpower = 1;
for(i=0; i < 3; i++)
  Xpower = X*Xpower;
```

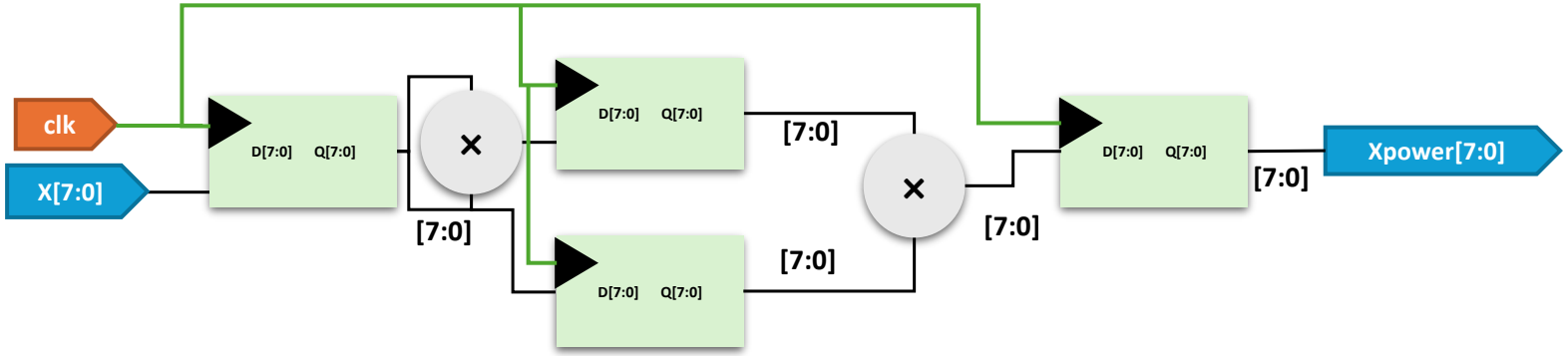


- Throughput after pipelining?
  - 1 task per cycle, 8 bits per cycle! (Improved!)

Design Tradeoffs: Pipelining

```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```

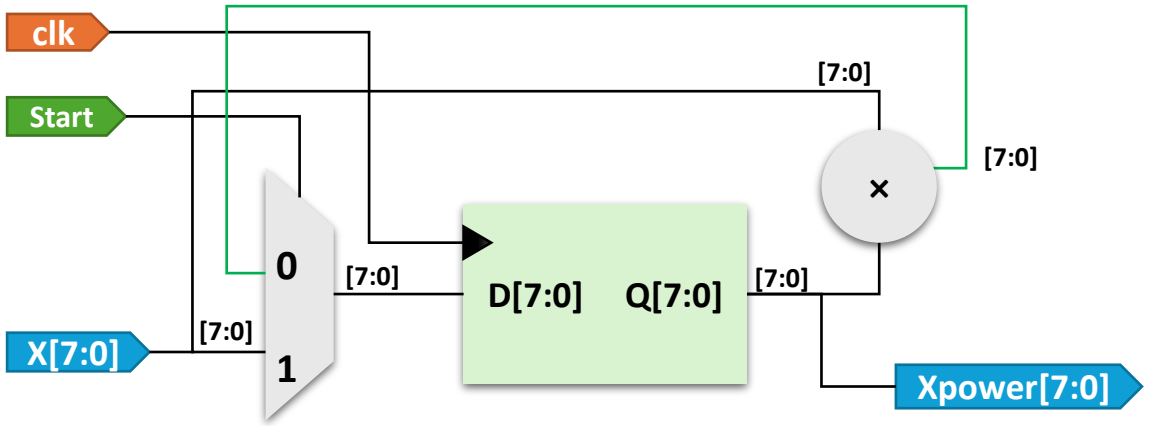


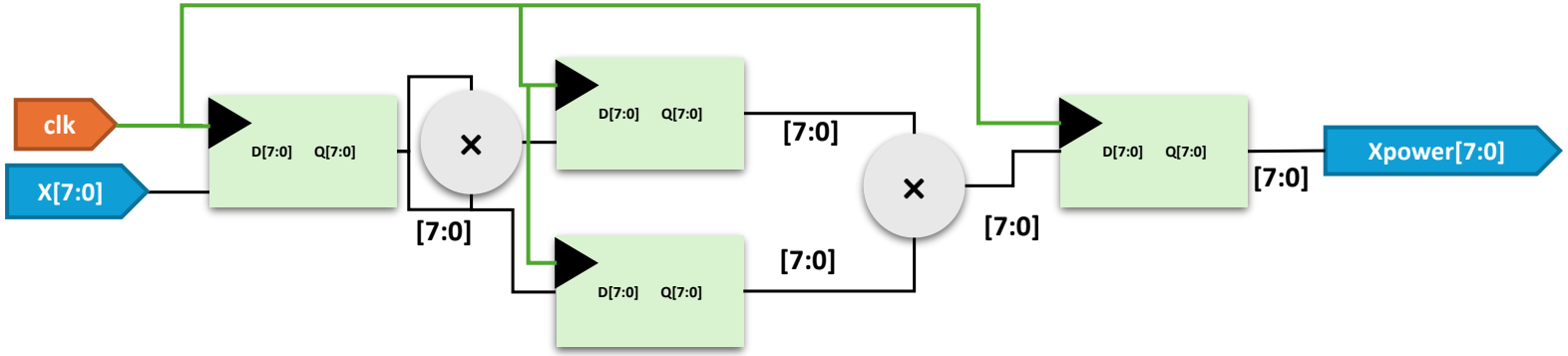


- Latency after pipelining?
  - 3 cycles! (Same!)

Design Tradeoffs: Pipelining

```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```

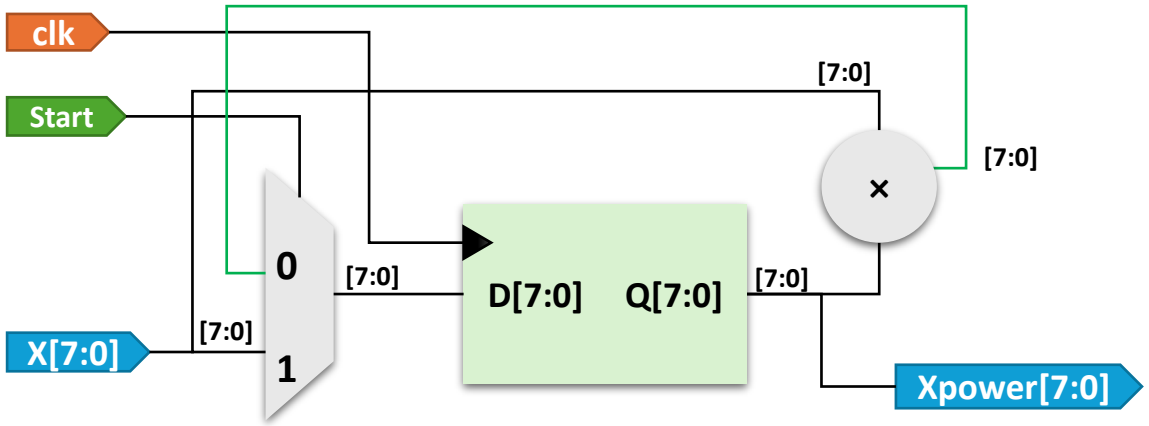


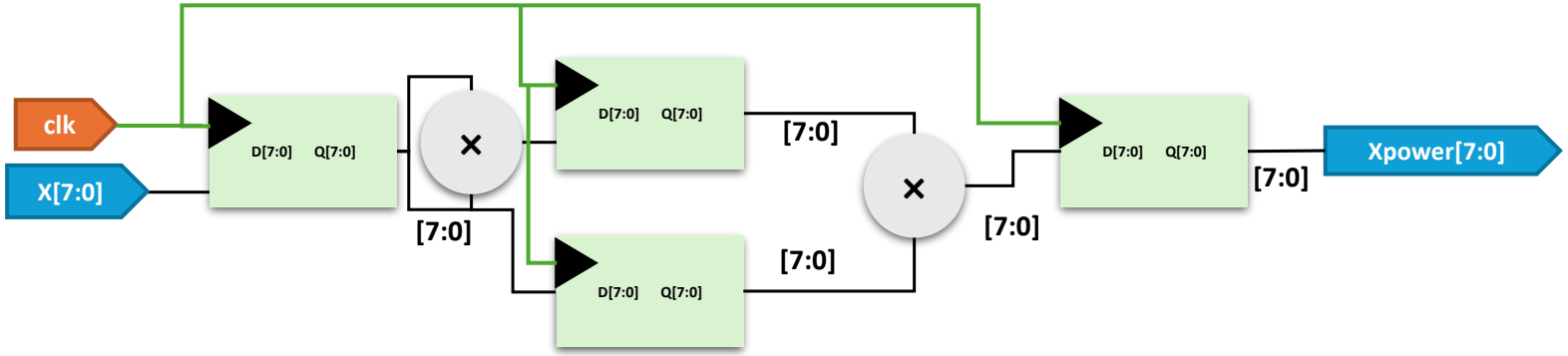


- Timing after pipelining?
  - Critical path still involves one multiplier delay (Same!)

Design Tradeoffs: Pipelining

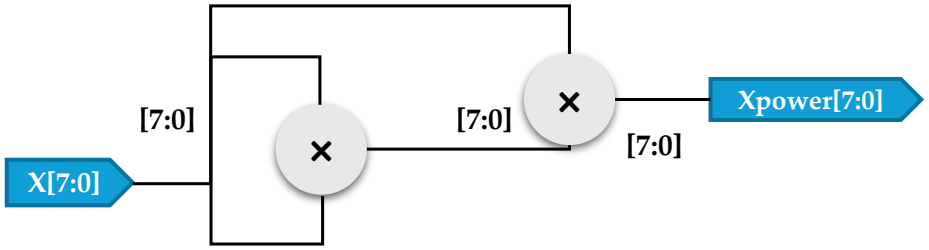
```
Xpower = 1;
for(i=0; i < 3; i++)
   Xpower = X*Xpower;
```





- Cost of pipelining?
  - More area! (Additional registers + Multiplier)

Design Tradeoffs: Single Cycle Design



- Throughput?
  - 1 sample per cycle or 8 bits per cycle!
- Latency?
  - 1 cycle (low latency!)
- Timing?
  - Clock period = 2 multipliers delay + clk2q + ts
  - Slower clock may *undermine* low latency!

How do we *evaluate* computers?

Instruction Timing

- What is the clock period of our datapath circuit?
  - Determined by longest path (lw)
  - Clock period: 800ps
  - Frequency: 1/800ps = 1.25 GHz
  - 1.25 billion instructions per second

- Canwe improve our datapath performance?
  - What does it mean to improve performance?
  - Quicker response time = one job finishes faster?
  - More jobs per unit time?
  - Longer battery life?

instr	Time taken per stage (in ps):					Total time taken:
	IF	ID	EX	MEM	WB	
add	200	100	200		100	600ps
beq	200	100	200			500ps
jal	200	100	200		100	600ps
lw	**200**	**100**	**200**	**200**	**100**	**800ps**
sw	200	100	200	200		700ps

Analogy: Transportation

- Consider two different vehicles:

	Sports Car	Bus
Passenger Capacity:	2	50
Travel Speed:	200 mph	50 mph
Gas Mileage:	5 mpg	2 mpg

- For a 50 mile trip (assuming instant return trips)...

	Sports Car	Bus
Travel Time:	15 min	60 min
Time for 100 passengers:	750 min (50 trips)	120 min (2 trips)
Gallons per passenger:	5 gallons	0.5 gallons

Measuring Performance in Computers

- We'll look at 3 different measures of computer performance:

Transportation Analogy	In a Computer
Trip Time	**Program execution time** (Example: Time to update display)
Time for 100 passengers	**Throughput** (Example: Number of server requests handled per hour)
Gallons per passenger	**Energy per task** (Example: How many movies you can watch per battery charge)

Optimizing Time Per Program

- The "Iron Law" of processor performance:

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Cycles}}{\text{Program}} \times \frac{\text{Time}}{\text{Cycle}}$$

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

- There are 3 components for optimizing the time it takes to execute a program

Optimizing Time Per Program

- The "Iron Law" of processor performance:

$$\frac{\text{Time}}{\text{Program}} = \boxed{\frac{\text{Instructions}}{\text{Program}}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

- Instructions per program determined by:
  - The task we're trying to do
  - The algorithm we implement, e.g. $\theta(N^2)$ or $\theta(N)$
  - The programming language we use
  - The compiler we use
  - The instruction set architecture (ISA) we use, e.g. x86 or RISC-V

Optimizing Time Per Program

- The "Iron Law" of processor performance:

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \boxed{\frac{\text{Cycles}}{\text{Instruction}}} \times \frac{\text{Time}}{\text{Cycle}}$$

- Average clock cycles per instruction (CPI) determined by:
  - The instruction set architecture (ISA).
  - Processor implementation and microarchitecture, e.g. our datapath design.

- Examples:
  - For the datapath we made, CPI = 1
  - For complex instructions (e.g. `strcpy`), CPI > 1
  - We'll see CPI < 1 processors later

Optimizing Time Per Program

- The "Iron Law" of processor performance:

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \boxed{\frac{\text{Time}}{\text{Cycle}}}$$

- Time per cycle (1/frequency) determined by:
  - Processor microarchitecture: Critical path through logic gates
  - Technology: How many transistors fit in a given space
  - Power budget: Lower voltages reduce transistor speed

Optimizing Time Per Program

- The "Iron Law" of processor performance:

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \boxed{\frac{\text{Time}}{\text{Cycle}}}$$

	Processor A	**Processor B**
# Instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate *f*	2.5 GHz	2 GHz
Execution time	1 ms	0.75 ms

Processor B is faster for this task, despite executing more instructions and having a slower clock rate!

Optimizing Energy Per Program

- There are 2 components for optimizing the energy it takes to execute a program

$$\frac{\text{Energy}}{\text{Program}} = \boxed{\frac{\text{Instructions}}{\text{Program}}} \times \frac{\text{Energy}}{\text{Instruction}}$$

- We already saw instructions per program earlier – it appears in this equation, too

Optimizing Energy Per Program

- There are 2 components for optimizing the energy it takes to execute a program

$$\frac{\text{Energy}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \boxed{\frac{\text{Energy}}{\text{Instruction}}}$$

- Energy per instruction $\propto CV^2$
  - C = capacitance
    - Depends on hardware technology and processor features
  - V = supply voltage

- To improve performance, we can reduce capacitance and voltage
  - Newer processors improve efficiency from Moore's law and lower supply voltage.
  - Reducing supply voltage too far causes "leakage power" - transistor switches don't fully turn off
  - In recent years, it's gotten harder to reduce supply voltage

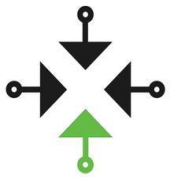
Optimizing Tasks Per Second

- The energy "Iron Law":

$$\underbrace{\frac{\text{Tasks}}{\text{Second}}}_{\text{Performance}} = \underbrace{\frac{\text{Joules}}{\text{Second}}}_{\text{Power}} \times \underbrace{\frac{\text{Tasks}}{\text{Joule}}}_{\text{Energy Efficiency}}$$

- Energy efficiency is a key metric in all computing devices.
  - If system is constrained by power, we need better energy efficiency to get more performance at the same power
  - For energy-constrained systems, we need better energy efficiency to prolong battery life

Practice Exercises …



Exercise 1

$$\text{CPU Execution Time for a program} = \text{CPU Clock Cycles for a program} \times \text{Clock Cycle Time}$$

$$\text{CPU Execution Time for a program} = \frac{\text{CPU Clock Cycles for a program}}{\text{Clock Rate (frequency)}}$$

- Example
  - Program runs in 10s on Computer A @ 2GHz
  - On B, it will run in 6s at a faster clock but will take 1.2 times more clock cycles!
  - What would be clock rate for Computer B?

Exercise 1

$$\text{CPU Execution Time for a program} = \text{CPU Clock Cycles for a program} \times \text{Clock Cycle Time}$$

$$\text{CPU Execution Time for a program} = \frac{\text{CPU Clock Cycles for a program}}{\text{Clock Rate (frequency)}}$$

- Solution
  - Cycles for Computer A = 10s × 2GHz = 20G cycles
  - Cycles for Computer B = 1.2 × 20G = 24G cycles
  - Clock Rate for Computer B = Cycles / Execution Time
    = (24G)/6 = 4 GHz

Instruction Performance

- Execution time also depends on number of instructions in a program!

$$\text{CPU Clock Cycles for a program} = \text{Instructions for a Program} \times \text{Average Clock Cycles per Instruction (CPI)}$$

- CPI (average) is one way of comparing different implementations of same ISA
  - Given same number of instructions per program

Exercise 2

$$\text{CPU Clock Cycles for a program} = \text{Instructions for a Program} \times \text{Average Clock Cycles per Instruction (CPI)}$$

- Example: Comparing two implementations of same ISA
  - Computer A has clock cycle of 250ps and CPI of 2.0 for a program
  - Computer B has clock cycle of 500ps and CPI of 1.2 for same program
  - Which one is faster for this program? How much?

- Solution
  - Clock cycles for Computer A = I × 2
  - Clock cycles for Computer B = I × 1.2
  - CPU Time for A = Clock cycles × Cycle Time = I × 2 × 250ps
  - CPU Time for B = Clock cycles × Cycle Time = I × 1.2 × 500ps
  - Computer B takes 1.2 times more execution time, i.e., it's 1.2 times slower!

CPU Performance Equation

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{CPU Time} = \frac{\text{Instructions Count} \times \text{CPI}}{\text{Clock Rate}}$$

$$\text{Time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instructions}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

Exercise 3

CPU Time = Instructions Count × CPI × Clock Cycle Time

Example: Comparing two code sequences!

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

Hardware Specifications
(By hardware designer)

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

Two alternative code sequences
(Options for compiler writer!)

- Which code sequence executes most instructions?
- What is CPI of each code sequence?
- Which will execute faster?
- Solution
  - Code Sequence 2 executes most instructions i.e., 6
  - Cycles for Code Sequence 1 = (2 × 1) + (1 × 2) + (2 × 3) = 10 cycles, CPI = 10/5 = 2
  - Cycles for Code Sequence 2 = (4 × 1) + (1 × 2) + (1 × 3) = 9 cycles, CPI = 9/6 = 1.5 (faster)
  - CPU Time for Code Sequence 1 = 10 × Clock Cycle Time
  - CPU Time for Code Sequence 2 = 9 × Clock Cycle Time (faster)
  - **Fewer instructions do not always mean faster execution !!!**

Exercise 4

- A given application written in Java runs 15 seconds on a desktop processor. A new Java compiler is released that requires only 0.6 as many instructions as the old compiler. Unfortunately, it increases the CPI by 1.1. How fast can we expect the application to run using this new compiler?

- Solution
  - CPU Time Old = 15s
  - Instructions Count Old = I
  - Instructions Count New = 0.6 × I
  - CPI Old = C
  - CPI New = 1.1 × C
  - CPU Time Old = 15s = I × C × Clock Cycle Time
  - CPU Time New = 0.6 × I × 1.1 × C × Clock Cycle Time
    = 0.66 × I × C × Clock Cycle Time
  - CPU Time New = 0.66 × CPU Time Old = 0.66 × 15s = 9.9s

Exercise 5

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.
  a. Which processor has the highest performance expressed in instructions per second?

- Solution (a)
  - Instructions per second = instructions per cycle × cycles per second
  - Instructions per second for P1 = (1/1.5) × 3GHz = 2G instructions/s
  - Instructions per second for P2 = (1/1) × 2.5GHz = 2.5G instructions/s
  - Instructions per second for P3 = (1/2.2) × 4GHz = 1.818G instructions/s
  - P2 has highest performance!

Exercise 5

CPU Time = Instructions Count × CPI × Clock Cycle Time

- Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.
  b. If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.

- Solution (b)
  - Execution time = Instruction count × CPI × Clock cycle time
  - Number of cycles = Execution Time × Clock Rate
  - Number of cycles for P1 = 10s × 3GHz = 30G cycles,
  - Number of cycles for P2= 25G cycles
  - Number of cycles for P3 = 40G cycles
  - Instructions count = (Execution Time × Clock Rate)/CPI
  - Instructions count for P1 = (10s × 3G)/1.5 = 20G instructions
  - Instructions count for P2 = (10s × 2.5G)/1.0 = 25G instructions
  - Instructions count for P3 = (10s × 4G)/2.2 = 18.18G instructions

Exercise 5

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.
  c. We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

- Solution (c)
  - 0.7 × CPU Time = Instructions Count × 1.2 × CPI × Clock Cycle Time
  - 1.2 / n = 0.7 → n = 1.2/0.7 = 1.714
  - The clock rate (for any processor) must be increased by 1.714 to achieve 30% reduction in execution time!

Exercise 6

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.
- Given a program with a dynamic instruction count of 1.0E6 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D
  - Which is faster: P1 or P2?
- Solution
  - CPU Time for P1 = $(1e6) \times (0.1 \times 1 + 0.2 \times 2 + 0.5 \times 3 + 0.2 \times 3) \times (1/2.5GHz)$
    $= 1.04 \times 10^{-3}$ seconds
  - CPU Time for P2 = $(1e6) \times (0.1 \times 2 + 0.2 \times 2 + 0.5 \times 2 + 0.2 \times 2) \times (1/3GHz)$
    $= 0.666 \times 10^{-3}$ seconds
  - P2 is faster!!

Exercise 6

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.
- Given a program with a dynamic instruction count of 1.0E6 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D
  - What is the global CPI for each implementation??

- Solution
  - Global CPI for P1 = $0.1 \times 1 + 0.2 \times 2 + 0.5 \times 3 + 0.2 \times 3 = 2.6$
  - Global CPI for P2 = $0.1 \times 2 + 0.2 \times 2 + 0.5 \times 2 + 0.2 \times 2 = 2$

Exercise 6

$$\text{CPU Time} = \text{Instructions Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.
- Given a program with a dynamic instruction count of 1.0E6 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D
  - Find the clock cycles required in both cases

- Solution
  - Clock Cycles for P1 = 1e6 × 2.6 = 2.6M cycles
  - Clock Cycles for P2 = 1e6 × 2 = 2M cycles

MIPS as Performance Measure

- MIPS: Million instructions per second

$$\text{MIPS} = \frac{\text{Instructions Count}}{\text{Execution Time} \times 10^6}$$

- MIPS is the rate of instructions execution
  - i.e., inverse of execution time!
- Limitations
  - Cannot compare computers with different ISAs as the instructions count and CPI would vary!
  - Varies between programs for same ISA!
- Alternatively,

$$\text{MIPS} = \frac{\text{Instructions Count}}{\frac{\text{Instructions Count} \times \text{CPI} \times 10^6}{\text{Clock Rate}}} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6}$$

MIPS vs Execution Time!

$$\text{MIPS} = \frac{\text{Instructions Count}}{\text{Execution Time} \times 10^6} \qquad \text{MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6}$$

- For a given program, consider:

Measurement	Computer A	Computer B
Instruction Count	10 billion	8 billion
Clock rate	4 GHz	4 GHz
CPI	1.0	1.1

  - Which computer has **higher MIPS rating**?
  - Solution
    - MIPS for Computer A = $4G/(1\times10^6) = 4\times10^3$
    - MIPS for Computer B = $4G/(1.1\times10^6) = 3.64 \times10^3$
    - Computer A has higher MIPS rating!

MIPS vs Execution Time!

$$\text{MIPS} = \frac{\text{Instructions Count}}{\text{Execution Time} \times 10^6} \qquad \text{MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6}$$

- For a given program, consider:

Measurement	Computer A	Computer B
Instruction Count	10 billion	8 billion
Clock rate	4 GHz	4 GHz
CPI	1.0	1.1

  - Which one is **faster**?
  - Solution
    - Execution Time for A = $(10G)/(4\times10^3 \times10^6)$ = 2.5s
    - Execution Time for B = $(8G)/(3.64 \times10^3\times10^6)$ = 2.198s
    - Computer B is **faster** despite having **lower MIPS rating**!

Execution Time is a more accurate measure of performance!

Amdahl's Law

- **Performance enhancement** possible by a given improvement is **limited** **by the** **amount that improved feature is used**!

$$\text{Execution time after improvement} = \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected}$$

- Example
  - Suppose a program runs in 100 seconds on a computer, with multiply operations responsible for 80 seconds of this time. How much do I have to improve the speed of multiplication if I want my program to run **five times** faster?

- Solution
  - 20 = (80/n) + 20
  - That is, there is *no amount* by which we can enhance-multiply to achieve a fivefold increase in performance, if multiply accounts for only 80% of the workload.

Thank You